Customer Segmentation and Sales Forecast

Transactions from Jan 2023 to Dec 2025

Student name: Syed Hamza Ali

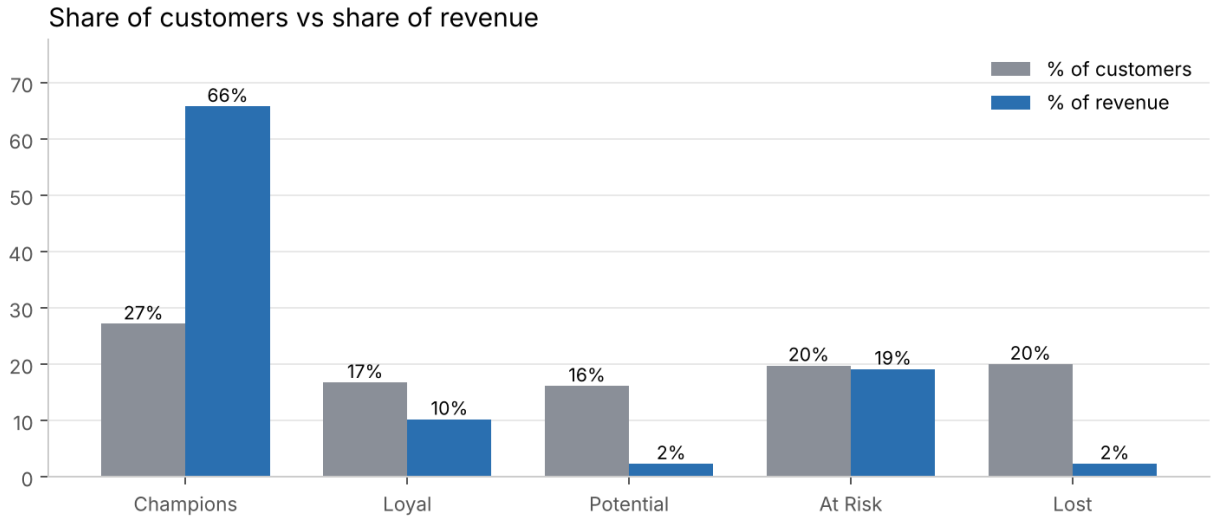
Key numbers

Metric	Value
Total sales	$1,069,669
Customers	798
Transactions	12,380
Average spend per customer	$1,340
Forecast Jan-Mar 2026	$105,406

Summary: Customers were grouped with RFM analysis (Recency, Frequency, Monetary). **Champions** are 27% of customers but bring 66% of revenue. **At Risk** customers (158 people) are worth $205,133 in past sales and should be contacted first. A trend + seasonality model predicts $105,406 in sales for the next 3 months.

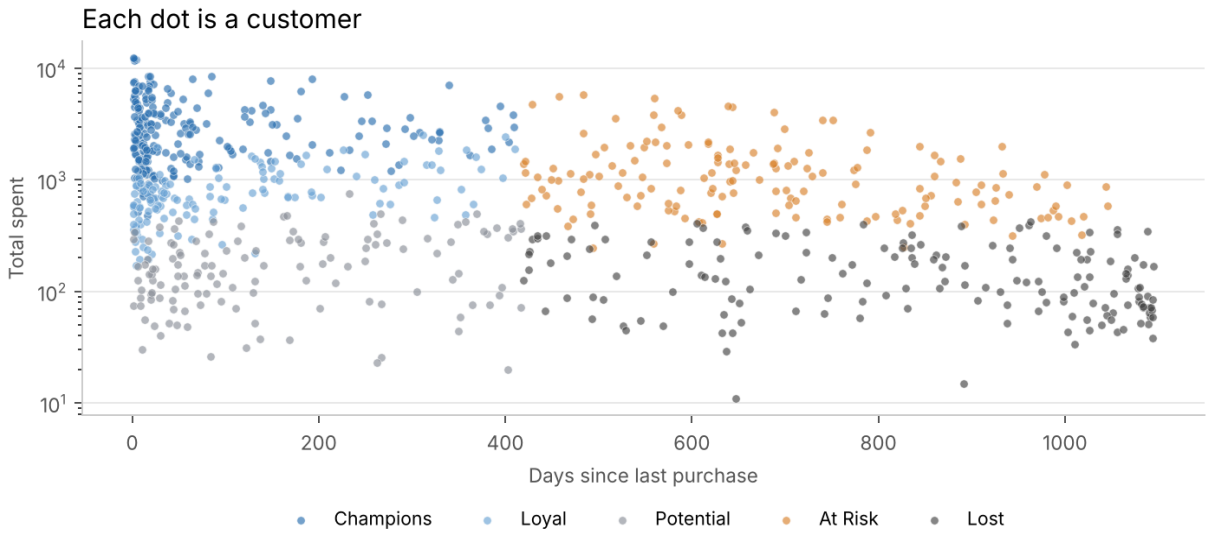
1. Customer segments (RFM)

Recency = days since last purchase, **F**requency = number of purchases, **M**onetary = total money spent. Each is scored 1-5 (5 = best) and the scores decide the segment.



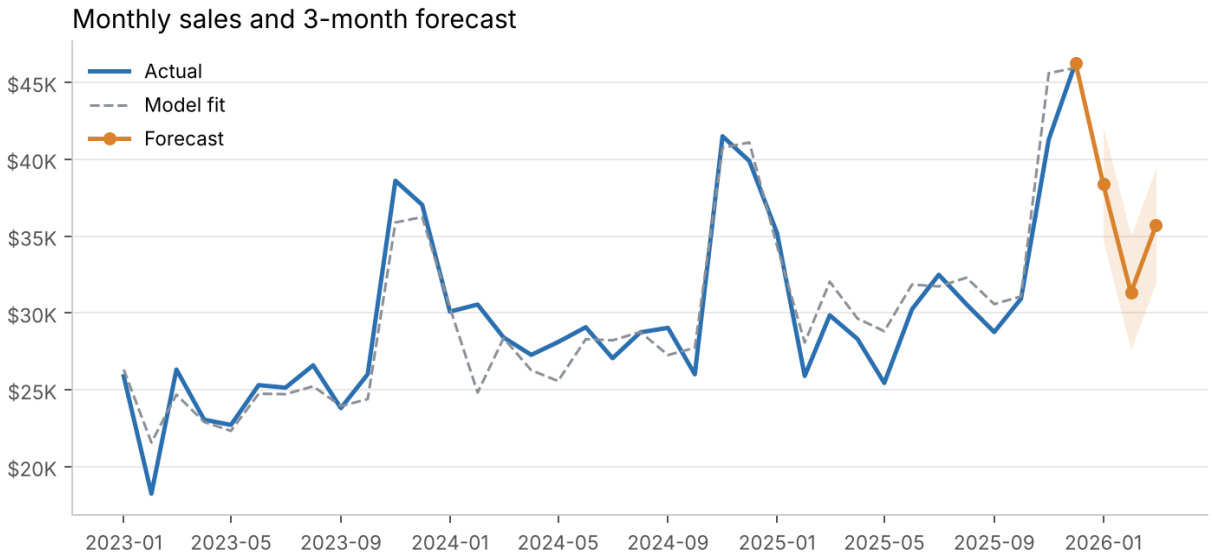
Segment	Customers	Avg days since buy	Avg purchases	Revenue
Champions	217	75	36.4	$704,272
Loyal	134	112	10.1	$109,667
Potential	129	159	2.7	$25,238
At Risk	158	683	15.4	$205,133
Lost	160	831	2.1	$25,359

2. Where customers sit



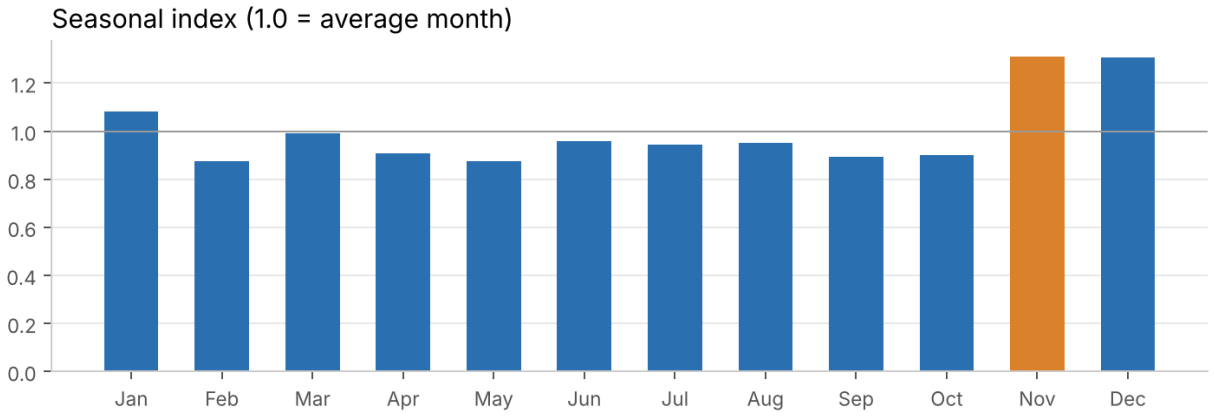
Customers on the left bought recently; those higher up spent more. At Risk and Lost customers are on the right side: they have not bought for a long time.

3. Sales forecast



Month	Forecast	Range (95%)
January 2026	$38,378	$34,669 - $42,088
February 2026	$31,318	$27,609 - $35,027
March 2026	$35,709	$32,000 - $39,419

Method: a straight-line trend (about $309 change per month) multiplied by a monthly seasonal index. On the past data the average error (MAPE) is 4.9%. In a back-test (hiding the last 3 months and predicting them) the error was 7.1%. Forecasts should be treated as a guide, not a promise.



4. Recommended actions

• Champions: reward them (early access, loyalty points) and ask for referrals.

• Loyal: offer bundles and upgrades to move them toward Champions.

• Potential: send a second-purchase offer within 30 days of the first order.

• At Risk: run a win-back campaign now (personal email, limited discount).

• Lost: low-cost reminder only; do not spend heavily on discounts.

• Plan stock and staffing around the seasonal peak in Nov.

Tools: Python, pandas, numpy, matplotlib, reportlab. Data is sample data created for this project.

Appendix A: Sales list

First 45 transactions by date, out of 12,380 ($1,069,669 total sales). The full list is created by the code as transactions.csv.

Customer ID	Date	Amount
670	2023-01-01	$58.15
262	2023-01-01	$88.24
792	2023-01-01	$73.51
38	2023-01-01	$25.92
456	2023-01-01	$81.72
134	2023-01-01	$39.50
555	2023-01-01	$33.61
786	2023-01-01	$86.99
493	2023-01-01	$44.11
500	2023-01-01	$162.20
733	2023-01-01	$148.70
72	2023-01-01	$62.46
80	2023-01-01	$83.73
372	2023-01-01	$101.20
695	2023-01-01	$99.50
548	2023-01-01	$134.26
733	2023-01-02	$119.62
500	2023-01-02	$228.15
614	2023-01-02	$54.75
562	2023-01-02	$95.62
144	2023-01-02	$169.70
103	2023-01-02	$61.28
548	2023-01-02	$93.39
113	2023-01-02	$36.24
467	2023-01-02	$67.49
602	2023-01-03	$55.20
454	2023-01-03	$59.38
80	2023-01-03	$61.56

387	2023-01-03	$42.95
616	2023-01-03	$85.19
780	2023-01-03	$45.40
553	2023-01-03	$50.25
687	2023-01-03	$64.51
61	2023-01-03	$21.61
628	2023-01-03	$38.37
569	2023-01-03	$19.13
321	2023-01-04	$67.59
485	2023-01-04	$48.91
791	2023-01-04	$47.17
28	2023-01-04	$75.77
677	2023-01-04	$80.04
582	2023-01-04	$46.41
468	2023-01-04	$45.49
456	2023-01-04	$70.33
38	2023-01-05	$60.31

Appendix B: Python source code

```
"""
Project 2: Customer Segmentation (RFM) and Sales Forecast
-----------------------------------------------------------
Steps:
  1. Create (or load) transaction data with customer ids
  2. RFM analysis: Recency, Frequency, Monetary -> customer segments
  3. Forecast next 3 months of sales (trend + seasonality)
  4. Charts + PDF report

Run:  python customer_forecast.py
Needs: pip install pandas numpy matplotlib reportlab
"""

import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import cm
from reportlab.lib import colors
from reportlab.platypus import (SimpleDocTemplate, Paragraph, Spacer, Image,
                                Table, TableStyle, PageBreak)

BASE = os.path.dirname(os.path.abspath(__file__))
DATA_FILE = os.path.join(BASE, "transactions.csv")
CHART_DIR = os.path.join(BASE, "charts")
PDF_FILE = os.path.join(BASE, "Customer_Segmentation_and_Forecast_Report.pdf")
os.makedirs(CHART_DIR, exist_ok=True)

BLUE, GREY, ORANGE = "#2a6fb0", "#8a8f98", "#d9822b"
SEGMENT_ORDER = ["Champions", "Loyal", "Potential", "At Risk", "Lost"]


# ---------------------------------------------------------------- 1. DATA
def create_sample_data(path, seed=7):
    """800 customers, 3 years of purchases with yearly seasonality and growth."""
    rng = np.random.default_rng(seed)
    start, end = pd.Timestamp("2023-01-01"), pd.Timestamp("2025-12-31")
    n_cust = 800
    rows = []
    for cid in range(1, n_cust + 1):
        # each customer has an activity level and a possible churn date
        rate = rng.gamma(2.0, 0.6)                    # purchases per month
        # 35% are existing customers at the start, others join steadily over 3 years
        if rng.random() < 0.35:
            first = start
        else:
            first = start + pd.Timedelta(days=int(rng.integers(0, 1060)))
        churn = first + pd.Timedelta(days=int(rng.exponential(600)))
        last_day = min(churn, end)
        days = (last_day - first).days
        if days <= 0:
            continue
        n_orders = max(1, rng.poisson(rate * days / 30))
        dates = first + pd.to_timedelta(rng.integers(0, days + 1, n_orders), unit="D")
        avg_amount = rng.lognormal(4.3, 0.5)
        for d in dates:
            season = 1.35 if d.month in (11, 12) else 1.0
            amount = max(5, rng.normal(avg_amount, avg_amount * 0.25)) * season
            rows.append((cid, d, round(amount, 2)))
    df = pd.DataFrame(rows, columns=["customer_id", "order_date", "amount"])
    df.sort_values("order_date").to_csv(path, index=False)


def load(path):
    df = pd.read_csv(path, parse_dates=["order_date"])
    df = df.drop_duplicates().dropna()
    df = df[df["amount"] > 0]
    print(f"{len(df):,} transactions, {df['customer_id'].nunique()} customers")
    return df


# ---------------------------------------------------------- 2. RFM
def rfm_analysis(df):
    snapshot = df["order_date"].max() + pd.Timedelta(days=1)
    rfm = df.groupby("customer_id").agg(
        recency=("order_date", lambda s: (snapshot - s.max()).days),
        frequency=("order_date", "count"),
        monetary=("amount", "sum"),
```

```
    )
    # score 1-5 (5 = best). Recency: smaller is better, so labels reversed.
    rfm["R"] = pd.qcut(rfm["recency"], 5, labels=[5, 4, 3, 2, 1]).astype(int)
    rfm["F"] = pd.qcut(rfm["frequency"].rank(method="first"), 5,
                       labels=[1, 2, 3, 4, 5]).astype(int)
    rfm["M"] = pd.qcut(rfm["monetary"].rank(method="first"), 5,
                       labels=[1, 2, 3, 4, 5]).astype(int)
    rfm["score"] = rfm["R"] + rfm["F"] + rfm["M"]

    def segment(row):
        if row["R"] <= 2 and row["F"] + row["M"] <= 4:
            return "Lost"
        if row["R"] <= 2:
            return "At Risk"          # used to buy a lot, but not recently
        if row["score"] >= 12:
            return "Champions"
        if row["score"] >= 9:
            return "Loyal"
        return "Potential"

    rfm["segment"] = rfm.apply(segment, axis=1)
    seg = rfm.groupby("segment").agg(
        customers=("recency", "count"),
        avg_recency=("recency", "mean"),
        avg_frequency=("frequency", "mean"),
        revenue=("monetary", "sum"),
    ).reindex(SEGMENT_ORDER).dropna(how="all")
    seg["revenue_share"] = seg["revenue"] / seg["revenue"].sum() * 100
    seg["customer_share"] = seg["customers"] / seg["customers"].sum() * 100
    return rfm, seg


# ------------------------------------------------------------ 3. FORECAST
def _fit_predict(monthly, months_ahead):
    """Fit trend x seasonality on `monthly`, return fitted values + predictions."""
    t = np.arange(len(monthly))
    slope, intercept = np.polyfit(t, monthly.values, 1)
    trend = intercept + slope * t
    ratio = monthly.values / trend
    seasonal = pd.Series(ratio, index=monthly.index.month).groupby(level=0).mean()
    seasonal = seasonal / seasonal.mean()
    fitted = trend * seasonal.reindex(monthly.index.month).values
    future_idx = pd.date_range(monthly.index[-1] + pd.offsets.MonthBegin(1),
                               periods=months_ahead, freq="MS")
    t_f = np.arange(len(monthly), len(monthly) + months_ahead)
    pred = (intercept + slope * t_f) * seasonal.reindex(future_idx.month).values
    return slope, seasonal, fitted, pd.Series(pred, index=future_idx)


def forecast(df, months_ahead=3):
    """Trend (straight line) x seasonal index, using complete months only."""
    monthly = df.set_index("order_date")["amount"].resample("MS").sum()
    if df["order_date"].max().day < 28:      # last month incomplete -> drop it
        monthly = monthly.iloc[:-1]

    slope, seasonal, fitted, future = _fit_predict(monthly, months_ahead)
    mape_in = np.mean(np.abs((monthly.values - fitted) / monthly.values)) * 100

    # honest test: hide the last 3 months, predict them, compare with reality
    _, _, _, back = _fit_predict(monthly.iloc[:-3], 3)
    actual = monthly.iloc[-3:]
    mape_back = np.mean(np.abs((actual.values - back.values) / actual.values)) * 100

    return {
        "monthly": monthly, "fitted": pd.Series(fitted, index=monthly.index),
        "future": future, "mape": mape_in, "mape_back": mape_back,
        "slope": slope, "resid_sd": np.std(monthly.values - fitted),
        "seasonal": seasonal,
    }


# ------------------------------------------------------------ 4. CHARTS
def style_axes(ax):
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)
    ax.spines["left"].set_color("#cccccc")
    ax.spines["bottom"].set_color("#cccccc")
    ax.tick_params(colors="#555555", labelsize=9)
    ax.grid(axis="y", color="#e8e8e8", linewidth=0.8)
    ax.set_axisbelow(True)


def money(x, _=None):
    return f"${x/1000:,.0f}K"


def make_charts(rfm, seg, fc):
```

```
    paths = {}

    # a) customers vs revenue share by segment
    fig, ax = plt.subplots(figsize=(8, 3.6))
    x = np.arange(len(seg))
    w = 0.38
    ax.bar(x - w/2, seg["customer_share"], w, color=GREY, label="% of customers")
    ax.bar(x + w/2, seg["revenue_share"], w, color=BLUE, label="% of revenue")
    for i, (c, r) in enumerate(zip(seg["customer_share"], seg["revenue_share"])):
        ax.text(i - w/2, c, f"{c:.0f}%", ha="center", va="bottom", fontsize=8.5)
        ax.text(i + w/2, r, f"{r:.0f}%", ha="center", va="bottom", fontsize=8.5)
    ax.set_xticks(x)
    ax.set_xticklabels(seg.index)
    ax.set_title("Share of customers vs share of revenue", loc="left", fontsize=12)
    ax.legend(frameon=False, fontsize=9)
    ax.set_ylim(0, max(seg["customer_share"].max(), seg["revenue_share"].max()) * 1.18)
    style_axes(ax)
    fig.tight_layout()
    paths["seg"] = os.path.join(CHART_DIR, "segments.png")
    fig.savefig(paths["seg"], dpi=180); plt.close(fig)

    # b) recency vs monetary scatter
    fig, ax = plt.subplots(figsize=(8, 3.8))
    palette = {"Champions": BLUE, "Loyal": "#6aa3d5", "Potential": GREY,
               "At Risk": ORANGE, "Lost": "#444444"}
    for name in SEGMENT_ORDER:
        d = rfm[rfm["segment"] == name]
        ax.scatter(d["recency"], d["monetary"], s=14, alpha=0.65,
                   color=palette[name], label=name, edgecolor="white", linewidth=0.3)
    ax.set_xlabel("Days since last purchase", fontsize=9, color="#555555")
    ax.set_ylabel("Total spent", fontsize=9, color="#555555")
    ax.set_yscale("log")
    ax.set_title("Each dot is a customer", loc="left", fontsize=12)
    ax.legend(frameon=False, fontsize=8.5, ncol=5, loc="upper center",
              bbox_to_anchor=(0.5, -0.2))
    style_axes(ax)
    fig.tight_layout()
    paths["scatter"] = os.path.join(CHART_DIR, "rfm_scatter.png")
    fig.savefig(paths["scatter"], dpi=180); plt.close(fig)

    # c) forecast
    fig, ax = plt.subplots(figsize=(8, 3.8))
    m, fut = fc["monthly"], fc["future"]
    ax.plot(m.index, m.values, color=BLUE, linewidth=2, label="Actual")
    ax.plot(fc["fitted"].index, fc["fitted"].values, color=GREY, linewidth=1.2,
            linestyle="--", label="Model fit")
    link_x = [m.index[-1]] + list(fut.index)
    link_y = [m.values[-1]] + list(fut.values)
    ax.plot(link_x, link_y, color=ORANGE, linewidth=2, marker="o", markersize=5,
            label="Forecast")
    ax.fill_between(fut.index, fut.values - 1.96*fc["resid_sd"],
                    fut.values + 1.96*fc["resid_sd"], color=ORANGE, alpha=0.15,
                    linewidth=0)
    ax.yaxis.set_major_formatter(plt.FuncFormatter(money))
    ax.set_title("Monthly sales and 3-month forecast", loc="left", fontsize=12)
    ax.legend(frameon=False, fontsize=9, loc="upper left")
    style_axes(ax)
    fig.tight_layout()
    paths["forecast"] = os.path.join(CHART_DIR, "forecast.png")
    fig.savefig(paths["forecast"], dpi=180); plt.close(fig)

    # d) seasonal index
    fig, ax = plt.subplots(figsize=(8, 2.9))
    s = fc["seasonal"]
    names = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
    ax.bar(names, s.values, color=[ORANGE if v == s.max() else BLUE for v in s.values], width=0.6)
    ax.axhline(1, color="#999999", linewidth=0.8)
    ax.set_title("Seasonal index (1.0 = average month)", loc="left", fontsize=12)
    style_axes(ax)
    fig.tight_layout()
    paths["season"] = os.path.join(CHART_DIR, "seasonal.png")
    fig.savefig(paths["season"], dpi=180); plt.close(fig)
    return paths


# ---------------------------------------------------------------- 5. PDF
def build_pdf(df, rfm, seg, fc, charts):
    ss = getSampleStyleSheet()
    H1 = ParagraphStyle("H1", parent=ss["Heading1"], textColor=colors.HexColor("#1d4f80"))
    H2 = ParagraphStyle("H2", parent=ss["Heading2"], textColor=colors.HexColor("#1d4f80"))
    body = ParagraphStyle("body", parent=ss["Normal"], fontSize=10.5, leading=15)
    doc = SimpleDocTemplate(PDF_FILE, pagesize=A4, leftMargin=2*cm, rightMargin=2*cm,
                            topMargin=2*cm, bottomMargin=2*cm,
                            title="Customer Segmentation and Sales Forecast")
    S = []

    def table(data, widths=None):
```

```
    t = Table(data, colWidths=widths)
    t.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#2a6fb0")),
        ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
        ("FONTSIZE", (0, 0), (-1, -1), 9.5),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#f2f5f9")]),
        ("GRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#cccccc")),
        ("ALIGN", (1, 0), (-1, -1), "RIGHT"),
        ("TOPPADDING", (0, 0), (-1, -1), 5), ("BOTTOMPADDING", (0, 0), (-1, -1), 5),
    ]))
    return t

total = df["amount"].sum()
n_cust = df["customer_id"].nunique()
top = seg.loc["Champions"] if "Champions" in seg.index else seg.iloc[0]
atrisk = seg.loc["At Risk"] if "At Risk" in seg.index else None
fut = fc["future"]

S += [Spacer(1, 3*cm),
      Paragraph("Customer Segmentation and Sales Forecast", ss["Title"]),
      Paragraph(f"Transactions from {df['order_date'].min():%b %Y} to "
                f"{df['order_date'].max():%b %Y}",
                ParagraphStyle("c", parent=body, alignment=1)),
      Spacer(1, 0.4*cm),
      Paragraph("<b>Student name:</b> Syed Hamza Ali", ParagraphStyle("nm", parent=body, alignment=1, fontSize=12)),
      Spacer(1, 1*cm)]
kpi = [["Metric", "Value"],
       ["Total sales", f"${total:,.0f}"],
       ["Customers", f"{n_cust:,}"],
       ["Transactions", f"{len(df):,}"],
       ["Average spend per customer", f"${total / n_cust:,.0f}"],
       [f"Forecast {fut.index[0]:%b}-{fut.index[-1]:%b %Y}", f"${fut.sum():,.0f}"]]
S += [Paragraph("Key numbers", H2), table(kpi, [9*cm, 5*cm]), Spacer(1, 0.6*cm)]
text = (f"<b>Summary:</b> Customers were grouped with RFM analysis (Recency, "
        f"Frequency, Monetary). <b>Champions</b> are {top['customer_share']:.0f}% "
        f"of customers but bring {top['revenue_share']:.0f}% of revenue. ")
if atrisk is not None:
    text += (f"<b>At Risk</b> customers ({atrisk['customers']:.0f} people) are worth "
             f"${atrisk['revenue']:,.0f} in past sales and should be contacted first. ")
text += (f"A trend + seasonality model predicts ${fut.sum():,.0f} in sales for the "
         f"next 3 months.")
S += [Paragraph(text, body), PageBreak()]

# RFM explanation + segments
S += [Paragraph("1. Customer segments (RFM)", H1),
      Paragraph("<b>R</b>ecency = days since last purchase, <b>F</b>requency = number "
                "of purchases, <b>M</b>onetary = total money spent. Each is scored "
                "1-5 (5 = best) and the scores decide the segment.", body),
      Spacer(1, 0.3*cm),
      Image(charts["seg"], 16.5*cm, 7.4*cm), Spacer(1, 0.3*cm)]
rows = [["Segment", "Customers", "Avg days since buy", "Avg purchases", "Revenue"]]
for name, r in seg.iterrows():
    rows.append([name, f"{r['customers']:.0f}", f"{r['avg_recency']:.0f}",
                 f"{r['avg_frequency']:.1f}", f"${r['revenue']:,.0f}"])
S += [table(rows, [3.5*cm, 2.8*cm, 3.8*cm, 3*cm, 3.5*cm])]
S.append(PageBreak())

S += [Paragraph("2. Where customers sit", H1),
      Image(charts["scatter"], 16.5*cm, 7.9*cm), Spacer(1, 0.3*cm),
      Paragraph("Customers on the left bought recently; those higher up spent more. "
                "At Risk and Lost customers are on the right side: they have not "
                "bought for a long time.", body), PageBreak()]

# forecast
S += [Paragraph("3. Sales forecast", H1),
      Image(charts["forecast"], 16.5*cm, 7.9*cm), Spacer(1, 0.3*cm)]
ft = [["Month", "Forecast", "Range (95%)"]]
for d, v in fut.items():
    ft.append([f"{d:%B %Y}", f"${v:,.0f}",
               f"${max(v - 1.96*fc['resid_sd'], 0):,.0f} - ${v + 1.96*fc['resid_sd']:,.0f}"])
S += [table(ft, [5*cm, 4*cm, 6*cm]), Spacer(1, 0.4*cm),
      Paragraph(f"<b>Method:</b> a straight-line trend (about "
                f"${fc['slope']:,.0f} change per month) multiplied by a monthly "
                f"seasonal index. On the past data the average error (MAPE) is "
                f"{fc['mape']:.1f}%. In a back-test (hiding the last 3 months and "
                f"predicting them) the error was {fc['mape_back']:.1f}%. "
                "Forecasts should be treated as a guide, not a promise.", body),
      Spacer(1, 0.4*cm), Image(charts["season"], 16.5*cm, 6*cm), PageBreak()]

# actions
S += [Paragraph("4. Recommended actions", H1)]
acts = [
    "Champions: reward them (early access, loyalty points) and ask for referrals.",
    "Loyal: offer bundles and upgrades to move them toward Champions.",
    "Potential: send a second-purchase offer within 30 days of the first order.",
    "At Risk: run a win-back campaign now (personal email, limited discount).",
    "Lost: low-cost reminder only; do not spend heavily on discounts.",
```

```
        f"Plan stock and staffing around the seasonal peak in "
        f"{['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec'][fc['seasonal'].idxmax()-1]}.",
    ]
    for a in acts:
        S += [Paragraph("• " + a, body), Spacer(1, 0.2*cm)]
    S += [Spacer(1, 0.6*cm),
          Paragraph("<i>Tools: Python, pandas, numpy, matplotlib, reportlab. "
                    "Data is sample data created for this project.</i>", body)]
    head = df.sort_values("order_date").head(45)
    SALES_ROWS = [["Customer ID", "Date", "Amount"]]
    for _, x in head.iterrows():
        SALES_ROWS.append([str(int(x.customer_id)), f"{x.order_date:%Y-%m-%d}", f"${x.amount:,.2f}"])
    SALES_WIDTHS = [4*cm, 4*cm, 4*cm]
    SALES_NOTE = (f"First 45 transactions by date, out of {len(df):,} "
                  f"(${df['amount'].sum():,.0f} total sales). The full list is created "
                  "by the code as transactions.csv.")

    # ---------- Appendix A: sales list ----------
    S.append(PageBreak())
    S.append(Paragraph("Appendix A: Sales list", H1))
    S.append(Paragraph(SALES_NOTE, body))
    S.append(Spacer(1, 0.3*cm))
    S.append(table(SALES_ROWS, SALES_WIDTHS))

    # ---------- Appendix B: source code ----------
    S.append(PageBreak())
    S.append(Paragraph("Appendix B: Python source code", H1))
    code_style = ParagraphStyle("code", fontName="Courier", fontSize=6.6, leading=8.2)
    with open(os.path.abspath(__file__), encoding="utf-8") as f:
        code = f.read().replace("\t", "    ").rstrip()
    from reportlab.platypus import Preformatted
    S.append(Preformatted(code, code_style))

    doc.build(S)


# ---------------------------------------------------------------- MAIN
if __name__ == "__main__":
    if not os.path.exists(DATA_FILE):
        create_sample_data(DATA_FILE)
    data = load(DATA_FILE)
    rfm, seg = rfm_analysis(data)
    print(seg.round(1))
    fc = forecast(data)
    print(fc["future"].round(0), f"MAPE={fc['mape']:.1f}% backtest={fc['mape_back']:.1f}%")
    charts = make_charts(rfm, seg, fc)
    build_pdf(data, rfm, seg, fc, charts)
    print("Done ->", PDF_FILE)
```